

CloudPulse

Design Document — Milestone 3

Project: Cloud Service Health Monitoring and Alert Dashboard

Team: Katha Patel • Ashwin S. Thankachan • Meet Patel • Pavithra Prasad

Repository: github.com/KathaPatel29/csye6300-summer26-group3-final-project

1. Design Patterns Used

CloudPulse retains exactly the nine patterns committed in Milestone 2: Factory Method, Adapter, Decorator, Strategy, State, Chain of Responsibility, Command, Observer, and MVC. Each pattern addresses a concrete design pressure in the implemented monitoring workflow. MVC is an architectural pattern rather than a Gang of Four pattern.

1.1 Factory Method

Problem solved: The scheduler and API workflows need HTTP and mock monitors without depending on their constructors.

How it is applied: MonitorCreator defines monitor creation, while HttpMonitorCreator and MockMonitorCreator return the matching ServiceMonitor implementation. HealthCheckScheduler resolves the appropriate creator from the service monitor type and receives a monitor through the common abstraction. New monitor families can therefore be added without rewriting scheduling clients.

1.2 Adapter

Problem solved: Monitored endpoints can return different response bodies even though the processing pipeline requires one stable result model.

How it is applied: HealthResponseAdapter defines the normalization contract. StandardHealthAdapter and LegacyHealthAdapter recognize their supported payload shapes and convert HTTP status, body, latency, and reachability information into HealthCheckResult. The monitoring and evaluation layers remain independent of external JSON formats.

1.3 Decorator

Problem solved: Logging, retry, and metrics behavior must be added around a monitor in combinations without producing many specialized subclasses.

How it is applied: MonitorDecorator wraps ServiceMonitor. LoggingMonitorDecorator, RetryMonitorDecorator, and MetricsMonitorDecorator each add one responsibility while preserving the same check contract. HealthCheckScheduler assembles the wrappers around the factory-created monitor, so the base monitor remains focused on probing a service.

1.4 Strategy

Problem solved: Different monitored services need different latency and failure thresholds while using the same evaluation workflow.

How it is applied: HealthEvaluationStrategy contains the classification operation. NormalHealthStrategy and StrictHealthStrategy compute a ServiceStatus from a health snapshot under different thresholds. HealthEvaluationService selects the configured strategy for each service. Strategy computes the observed classification; it does not own lifecycle transitions.

1.5 State

Problem solved: A service can be Unknown, Healthy, Degraded, Down, or Recovered, and its legal next status depends on its current lifecycle state.

How it is applied: ServiceState defines state-specific transition behavior. Concrete Unknown, Healthy, Degraded, Down, and Recovered state objects determine the next state from the classification produced by Strategy. ServiceStateMachine delegates to the current state, keeping transition and recovery rules out of controllers and evaluators.

1.6 Chain of Responsibility

Problem solved: A health result must pass validation, availability, latency, status-change, and alert stages in a consistent order without one monolithic processor.

How it is applied: ResultHandler stores the next handler reference. ResponseValidationHandler, AvailabilityHandler, LatencyHandler, StatusChangeHandler, and AlertCreationHandler each perform one focused step before delegating. HealthCheckProcessor owns the assembled chain, allowing stages to be tested and changed independently.

1.7 Command

Problem solved: Manual checks, scheduled checks, monitoring toggles, and alert actions need consistent execution boundaries that are separate from HTTP controllers and schedulers.

How it is applied: MonitoringCommand defines command execution. Concrete commands encapsulate running a health check, enabling or disabling monitoring, acknowledging an alert, and resolving an alert together with the identifiers and collaborators they require. Controllers and scheduled tasks invoke commands rather than duplicating workflow coordination.

1.8 Observer

Problem solved: A committed status transition may need multiple independent reactions, and one subscriber failure must not break the others.

How it is applied: StatusEventPublisher maintains EventSubscriber registrations. ObserverRegistrar connects the Spring-managed dashboard, alert, and event-history subscribers. StatusChangeHandler publishes only after a successful transaction commit, and the publisher isolates subscriber exceptions so future reactions can be added without changing the transition workflow.

1.9 MVC

Problem solved: Presentation, HTTP request handling, domain rules, and persistence need clear boundaries so the React UI and backend can evolve independently.

How it is applied: React components and REST clients form the View. Spring REST controllers form the Controller layer and delegate to commands, services, and repositories. MonitoredService, HealthCheckResult, StatusEvent, Alert, supporting domain services, and JPA repositories form the Model. JSON REST requests connect the browser to the backend.

2. System Architecture

CloudPulse is a full-stack monitoring application with a React browser client, a layered Spring Boot backend, relational persistence, and three controllable demo health services. The browser communicates only with the backend. The backend owns monitoring, classification, lifecycle transitions, alerting, history, and persistence, and it performs outbound HTTP checks against registered health URLs.

2.1 Runtime Architecture

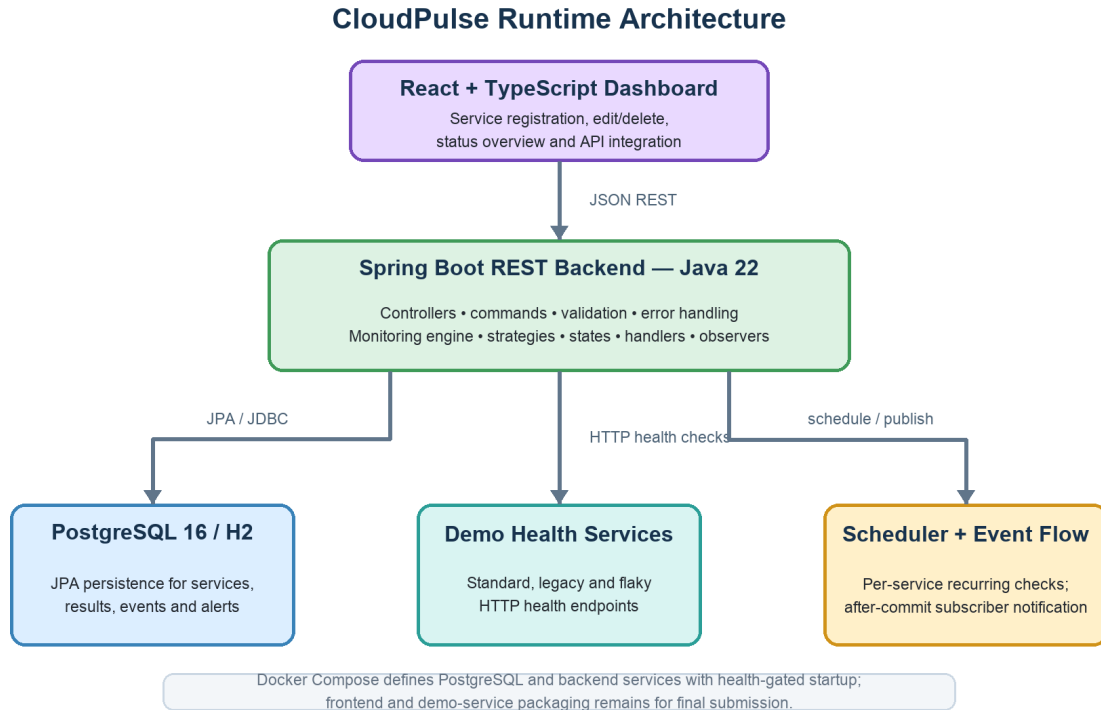


Figure 1. CloudPulse runtime components and principal data-flow boundaries.

2.2 Frontend — React 19, TypeScript, and Vite

The frontend is a single-page dashboard that uses JSON REST calls to the Spring Boot API. The implemented Milestone 3 slice lists monitored services, shows status badges and summary counts, and supports service registration, editing, and protected deletion. Loading, empty, and failure states are presented in the UI. Alert, history, policy, manual-check, and monitoring-toggle clients exist at the API boundary but are not yet all exposed as dashboard controls.

- App coordinates the service-management screen and summary state.
- ServiceList and related components render services and initiate edit or delete actions.
- ServiceForm validates and submits registration or update requests.
- Typed API modules isolate REST request details from visual components.

2.3 Backend — Spring Boot 3.3.4 and Java 22

The backend is a layered Spring application. REST controllers validate request DTOs and delegate to application services or commands. The monitoring packages create and decorate monitors, normalize remote responses, evaluate health, apply lifecycle rules, execute the processing chain, persist domain changes, and publish committed status events. Controllers contain transport concerns; domain and application packages contain monitoring behavior.

- Service and alert controllers expose registration, lifecycle, and operator actions.
- Evaluation-policy, health-check-history, and status-event-history controllers expose the Milestone 3 read and configuration APIs.
- Global exception handling returns consistent JSON for validation errors, missing resources, unsupported methods, and protected-delete conflicts.

- OpenAPI/Swagger provides an executable view of endpoints not yet surfaced in the React dashboard.

2.4 Persistence — Spring Data JPA, H2, and PostgreSQL 16

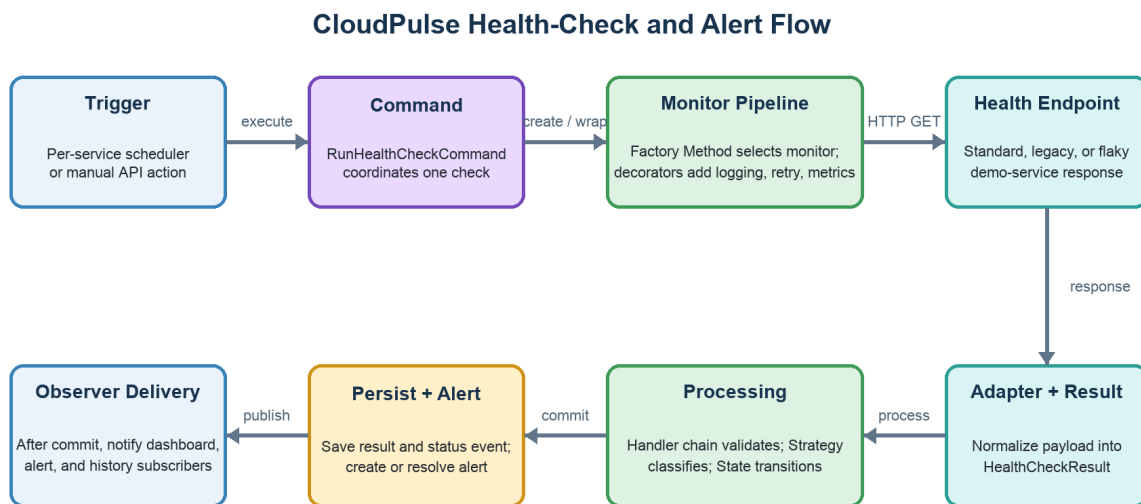
JPA repositories persist monitored services, health-check results, lifecycle events, and alerts. The development and test profile uses H2 for fast local feedback. The Docker profile is configured to connect the backend to PostgreSQL 16. Service deletion is rejected when historical monitoring records exist, preserving referential and audit consistency. No cache is present or required for the current student-scale workload.

2.5 Demo Health Services

Three small Spring Boot services supply repeatable monitoring targets. The Standard service returns the expected health format, the Legacy service exercises response adaptation, and the Flaky service produces variable health behavior for retry, latency, state-transition, and alert demonstrations. They run separately on ports 8081, 8082, and 8083 at this milestone.

2.6 Scheduling, Monitoring, and Event Flow

HealthCheckScheduler maintains one recurring task per enabled service and reschedules it when the interval changes. Each task invokes a command that obtains a factory-created monitor wrapped with logging, retry, and metrics decorators. After the response is adapted, the handler chain performs validation and processing. Strategy determines the observed classification, State owns the transition, repositories commit the result, and Observer subscribers react after commit.



The alert and event branches occur only when the processed result changes lifecycle state or meets an alert rule.

Figure 2. End-to-end health-check, persistence, alert, and event flow.

2.7 Principal Data Flows

Service registration or update: Browser request → ServiceController → validation → service repository → JSON response. Interval changes cause the scheduler to cancel and recreate the corresponding recurring task.

Scheduled or manual check: Scheduler or REST action → monitoring command → creator → decorated monitor → configured HTTP endpoint → adapter → HealthCheckResult → processing chain.

Classification and transition: HealthEvaluationService selects NORMAL or STRICT Strategy → Strategy computes ServiceStatus → ServiceStateMachine asks the current State to transition → service status and timestamps are updated.

Persistence and notification: Repositories save results, status events, service state, and alerts → transaction commits → StatusEventPublisher notifies registered subscribers with per-subscriber failure isolation.

3. Component and Service Breakdown

Package or service	Purpose	Patterns used and rationale
frontend/src	React dashboard, forms, service cards, summary state, and typed REST clients.	MVC View — presents data and forwards user intent without backend business rules.
Controller	Service, alert, policy, health-result-history, and status-event-history endpoints.	MVC Controller — translates HTTP requests and delegates application work.
Scheduler	Creates, cancels, and reschedules a recurring check for each enabled service.	Uses Factory Method, Decorator, and Command to compose scheduled execution.
Monitor	ServiceMonitor abstraction, HTTP/mock implementations, monitor creators, and wrappers.	Factory Method selects implementations; Decorator layers cross-cutting behavior.
Adapter	Recognizes standard or legacy endpoint bodies and creates a common result.	Adapter shields the domain workflow from external payload differences.
Evaluation	Applies NORMAL or STRICT thresholds to produce an observed classification.	Strategy makes the policy replaceable per monitored service.
Lifecycle	Represents Unknown, Healthy, Degraded, Down, and Recovered transition rules.	State localizes lifecycle behavior and recovery decisions.
Handler	Runs validation, availability, latency, status-change, and alert stages.	Chain of Responsibility gives each stage one concern and explicit ordering.
Command	Encapsulates checks, monitoring toggles, acknowledgement, and resolution actions.	Command separates an action request from its execution workflow.
Observer	Registers and notifies dashboard, alert, and event-history subscribers after commit.	Observer decouples transition producers from independent reactions.
Model and repository	Defines persisted services, results, events, alerts, and repository access.	MVC Model plus the persistence boundary used by domain workflows.
Demo services	Provide standard, legacy, and variable HTTP health responses.	External test targets that exercise Adapter and the full monitoring pipeline.
Docker runtime	Starts PostgreSQL, verifies database health, and launches the backend profile.	Deployment infrastructure; it does not introduce another design pattern.

4. Technology Stack

4.1 Frontend

Technology	Purpose
React 19	Component-based dashboard rendering and local UI state.
TypeScript 5.7	Typed component props, domain models, and REST clients.
Vite 6	Frontend development server and production build pipeline.
Tailwind CSS 4	Responsive layout, status badges, forms, and feedback states.

4.2 Backend

Technology	Purpose
Java 22	Application language for the backend and demo health services.
Spring Boot 3.3.4	Dependency injection, configuration, scheduling, profiles, and runtime packaging.
Spring Web / Validation	REST controllers, JSON request handling, DTO validation, and HTTP responses.
Spring Data JPA / Hibernate	Repository abstraction, entity mapping, and relational persistence.
Jackson	JSON serialization and standard/legacy health-response parsing.
Springdoc OpenAPI 2.6	Swagger UI and API contract discovery.
Maven	Backend build, dependency management, packaging, and test execution.

4.3 Data, Infrastructure, and Verification

Technology	Purpose
H2	Fast local development and automated-test database profile.
PostgreSQL 16	Relational database used by the Docker runtime profile.
Docker Compose	Configuration for health-gated PostgreSQL and backend startup.
JUnit 5, Mockito, MockMvc	Unit, controller, scheduler, workflow, and Observer integration verification.
GitHub Actions	Repository automation for backend and frontend verification.
npm	Frontend dependency installation and production build.

5. Current Milestone Boundary and Quality

5.1 Implemented Boundary

- The backend exposes service, alert, evaluation-policy, health-check-history, and status-event-history APIs.
- The React application completes the service-management dashboard slice, including registration, edit, and protected delete.
- Per-service scheduling, decorated monitoring, result processing, transition persistence, alert logic, and live Observer publication are integrated.
- Docker Compose defines PostgreSQL and backend services with health-gated startup. The frontend and three demo services remain separate processes.

5.2 Testability and Reliability

- Pattern boundaries keep monitors, adapters, strategies, states, handlers, commands, observers, and controllers independently testable.
- The latest packaged Surefire reports contain 77 backend tests with zero recorded failures and errors.
- Per-service scheduled tasks are cancelled or rescheduled when configuration changes.
- Retry behavior is bounded, and scheduler exceptions are isolated per service.
- Status events publish after transaction commit, and subscriber exceptions are isolated.
- Services with monitoring history cannot be deleted accidentally; callers receive a conflict response.

5.3 Scope Constraints

- CloudPulse is a single-environment educational monitor rather than a commercial observability platform.
- Authentication, production notifications, distributed tracing, multi-tenancy, and large-scale caching remain outside the current scope.
- The dashboard currently covers service management; remaining API workflows are planned for the final submission UI.